map(parseInt)

Why it returns [1, NaN, NaN]

map-parseint.js

```
1 ["1", "2", "3"].map(parseInt);
```

output `1` `NaN` `NaN`

Why it happens

01 map passes 3 args

Every call gets (value, index, array), not just the value.

02 parseInt takes 2 args

parseInt(string, radix). The radix is the number base.

03 Index becomes radix

So the index 0, 1, 2 is used as the base for each item.

04 Invalid base = NaN

Base 1 does not exist, and "3" is not a binary digit.

What map really calls

what-map-really-does.js

```
1 parseInt("1", 0); // 1 (0 = default base 10)
2 parseInt("2", 1); // NaN (base 1 is invalid)
3 parseInt("3", 2); // NaN (binary has only 0, 1)
```

The fix

Pass only the value

fixes.js

```
1 ["1", "2", "3"].map(Number); // [1, 2, 3]
2 ["1", "2", "3"].map((s) => parseInt(s, 10)); // [1, 2, 3]
```

expression	result	why
`parseInt("42px")`	42	stops at the first non-digit
`Number("42px")`	NaN	whole string must be numeric
`parseInt("0.5")`	0	integers only
`parseInt("ff", 16)`	255	base 16 (hex)
`["10","10","10"].map(parseInt)`	[10, NaN, 2]	"10" in base 2 is 2

Traps to mention

- **Always pass the radix** to parseInt: `parseInt(s, 10)`.
- **Same trap with other callbacks.** Any function with optional args can misread the index.
- **Number vs parseInt:** Number is strict, parseInt reads digits until it hits a letter.

Your 20-second answer

"map calls the callback with the value AND the index. parseInt takes a second argument, the radix, so the index becomes the number base. Base 0 means default, so '1' works, but base 1 is invalid and '3' is not a binary digit, so both give NaN. The fix is map(Number) or an arrow function."